

Patrones y estándares de acceso a datos



Repositorios de Información

Escuela de Ingeniería Informática

2026-2027

Universidad de Oviedo

Atributos de calidad para el software

Diagnóstico del proyecto inicial

Patrones

Layers

Fachada

Factoría Simple

Section 1

Atributos de calidad para el software

- **Mantenibilidad:** facilidad para localizar y realizar cambios sin tener que modificar partes no relacionadas del sistema.



- **Reusabilidad** Capacidad de utilizar un componente en diferentes partes del código sin tener que modificarlo. [Bibliotecas \(Math\), ...](#)
- **Extensibilidad** Facilidad de adaptar el software a cambios de especificación afectando lo menos posible al software ya existente.
 - Tipo mediante atributo: `class Vehicle { String vType ....`
 - Tipo mediante clase: `VehicleType`.
 - Nuevo tipo de vehículo: ¿qué cambios requiere cada uno?
- **Escalabilidad** Capacidad para manejar el crecimiento del sistema (volumen de datos, usuarios, complejidad ...),
[Bloqueos vs mvcc, Versiones de Twitter](#) ¹

¹<https://medium.com/@siddhantsambit/scalability-twitters-journey-16ed5af2e01b>

Mantenibilidad	¿Es fácil cambiar lo que ya existe?
Reusabilidad	¿Puedo utilizar lo que ya tengo en otro contexto?
Extensibilidad	¿Es fácil incorporar algo nuevo?
Escalabilidad	¿Qué ocurre cuando el sistema crece?

Section 2

Diagnóstico del proyecto inicial

El proyecto CWS0

Estructura de paquetes por actores:

Paquete contiene menús de texto y subpaquete action.

Clases Action implementan las acciones para realizar la acción.

- ▼  cws0
 - ▼  src
 - ▼  uo.ri
 - >  conf
 - ▼  cws.application.ui
 - >  cashier
 - >  foreman
 - ▼  manager
 - ▼  mechanic
 - ▼  action
 - >  AddMechanicAction.java
 - >  DeleteMechanicAction.java
 - >  ListAllMechanicsAction.java
 - >  ListMechanicAction.java
 - >  UpdateMechanicAction.java
 - >  MechanicMenu.java
 - >  MainMenu.java

Diagrama de clases de Gestión de Mecánicos

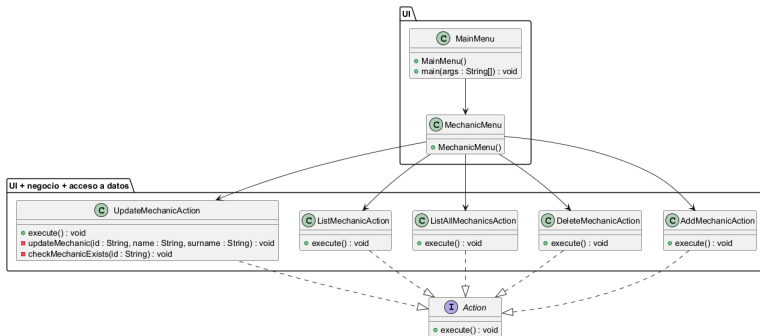
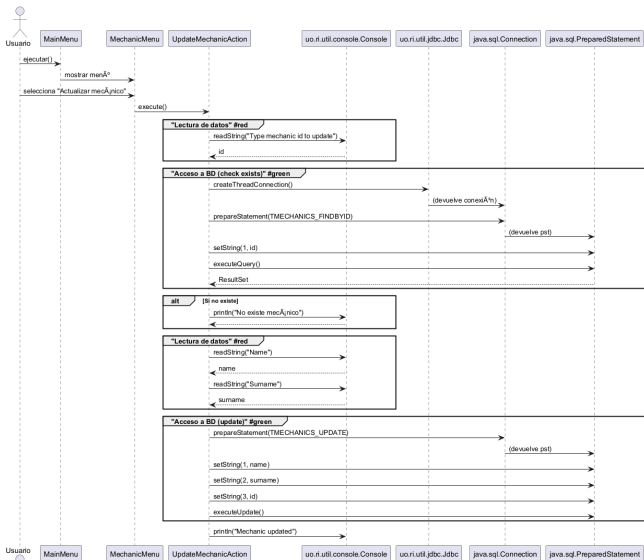


Diagrama de secuencia de Gestión de Mecánicos



```

public class UpdateMechanicAction
    implements Action {

    private static final String FINDBYID =
        "select * from TMechanics where id = ?";
    private static final String UPDATE =
        "update TMechanics set name=?, surname=?, "
        + "version = version + 1, updatedat = ?"
        + "where id = ?";

    @Override
    public void execute()
        throws BusinessException {
        // Get info
        String id = Console
            .readString("mechanic id ");

        // check mechanic exists
        checkMechanicExists(id);
        // Ask for new data
        String name = Console.readString("Name");
        String sname = Console.readString("Surname");

        // update
        updateMechanic(id, name, sname);

        // Print result
        Console.println("Mechanic updated");
    }
}

```

```

private void updateMechanic(String id,
    String name, String surname) {

    try (Connection c = DriverManager
        .getConnection(URL, USER, PASS);
        PreparedStatement pst = c
            .prepareStatement(UPDATE)) {
        pst.setString(1, name);
        pst.setString(2, surname);
        pst.setTimestamp(3, new Timestamp(...));
        pst.setString(4, id);
        pst.executeUpdate();
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}

private void checkMechanicExists(String id)
    throws BusinessException {
    try (Connection c = DriverManager
        .getConnection(URL, USER, PASS);
        PreparedStatement pst = c
            .prepareStatement(FINDBYID)) {
        pst.setString(1, id);
        try (ResultSet rs = pst.executeQuery())
            if (!rs.next()) {
                throw new BusinessException("...");
            }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
}

```

Mantenibilidad

Evaluación de CWS0

Causas del problema:

- Código enmarañado: responsabilidades mezcladas en las clases.
- Código disperso: una responsabilidad o regla en varias clases.

Ejemplos de cambios:

- Acceso a datos (Hsqldb → Oracle, JDBC → Hibernate).
- Interfaz de usuario
- Reglas negocio (Contrato vigente: no modificar/eliminar mecánico)

Consecuencias:

- Múltiples motivos para modificar una misma clase.
- Un cambio puede afectar a varias clases.
- Cambios más difíciles de localizar y verificar.

Reusabilidad

Evaluación de CWS0

Causas del problema:

- Lógica útil para varios casos encerrada en cada caso de uso.

Ejemplos:

- UpdateMechanic usa Scanner → difícil reutilizarlo desde una GUI.
- UpdateMechanic::checkMechanicExists → difícil reutilizarlo.
- Acceso a base de datos dentro de Action.

Consecuencias:

- La lógica no está disponible como componente reutilizable.
- Es necesario modificar el diseño para poder reutilizarla.

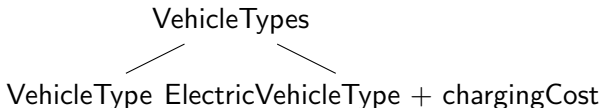
Extensibilidad

Causas del problema:

- Código acoplado a una implementación concreta.
- Variantes resueltas mediante condiciones.

Ejemplos: Añadir nueva especificación: "Los vehículos eléctricos tienen un coste de carga adicional".

1. Modificar `VehicleType` incorporando nueva variante del cálculo.
2. Extender el modelo con una nueva clase.



Consecuencias:

- Añadir una nueva variante requiere modificar código existente.
- Los cambios pueden afectar a funcionalidades ya existentes.

Escalabilidad

Causas del problema:

- Dependencia de recursos limitados.

Ejemplo:

- Cada Action crea su propia conexión a la BBDD.
- Aumenta el número de usuarios concurrentes.
- Las conexiones se convierten en un cuello de botella.
- ¿Podemos incorporar un *connection pool* sin modificar gran parte del código existente?

Consecuencias:

- Mayor tiempo de respuesta.
- Degradación del rendimiento al crecer el sistema.
- Cambios importantes para adaptar el sistema al crecimiento.

Section 3

Patrones

¿Qué es un patrón de diseño?

Un patrón describe un problema que ocurre una y otra vez, así como la solución a ese problema.

Christopher Alexander

En software:

Los patrones arquitectónicos definen la estructura y organización general del sistema.

Los patrones de diseño son descripciones de clases y objetos relacionados para resolver problemas de diseño generales.

Gamma, Helm, Johnson y Vlissides

Un patrón es una respuesta a un problema de diseño.

De un problema de diseño a un patrón

1. **Detectamos un problema de diseño**
2. Identificamos el **atributo de calidad** que queremos mejorar
3. Determinamos el **cambio que queremos facilitar**
4. Buscamos un **patrón de diseño** que nos ayude
5. Evaluamos qué **ganamos y qué coste introducimos**

Problema → **Objetivo** → **Patrón** → **Consecuencias**

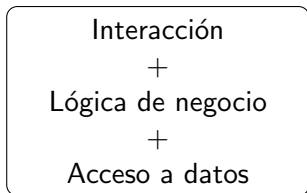
Section 4

Layers

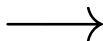
Problemas actuales y atributos de calidad afectados

Problema: las responsabilidades de la aplicación están **mezcladas**, dificultando su evolución.

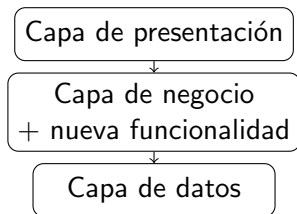
Actual



Responsabilidades acopladas



Objetivo



Responsabilidades separadas

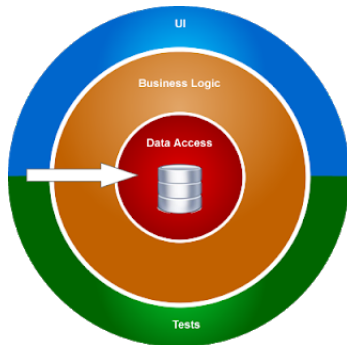
Calidad afectada: mantenibilidad, reusabilidad y extensibilidad.

Objetivo: separar las responsabilidades.

Patrón arquitectónico: Layers

Idea fundamental

Organizar el sistema en **capas**, asignando a cada una responsabilidades relacionadas y estableciendo dependencias controladas entre ellas.



Selección de capas

Presentación

Interacción con el usuario



Negocio

Reglas y lógica del problema



Acceso a datos

Persistencia de los datos

Cada capa utiliza **únicamente** los servicios proporcionados por la capa inferior.

¿Qué responsabilidad tiene cada capa?

■ **Presentación**

- Interacción con el usuario.
- Recoge las solicitudes.
- Muestra los resultados.

■ **Negocio**

- Implementa las reglas del problema.
- Coordina las operaciones del sistema.

■ **Acceso a datos**

- Recupera y almacena datos.
- Oculta los detalles de persistencia.

¿Qué conseguimos?

- **Separación de responsabilidades**

Cada capa tiene un propósito concreto.

- **Mantenibilidad**

Los cambios quedan más localizados dentro de cada capa.

- **Reusabilidad**

La lógica de negocio puede ser utilizada por distintos clientes.

Costes y limitaciones

- No siempre es posible una separación estricta entre capas.
- Las capas adicionales pueden introducir **coste de rendimiento**.
- Una arquitectura de capas estricta obliga establecer y limitar las dependencias entre capas.
- Para sistemas pequeños, puede introducir **complejidad innecesaria**.

Separando AddMechanic en capas

Antes

AddMechanic

- Interacción usuario
- Valida los datos
- Chequea reglas negocio
- Gestiona conexiones
- Acceso a BBDD

Después

AddMechanic UI

Deja de conocer detalles de persistencia.



Mechanic Service

Lógica de negocio no conoce UI.



Mechanic Data Access

El acceso a datos queda aislado.

Cada responsabilidad se sitúa en una capa diferente.

Las dependencias siguen una única dirección:

AddMechanic → MechanicService → MechanicDAO

Estructura de paquetes

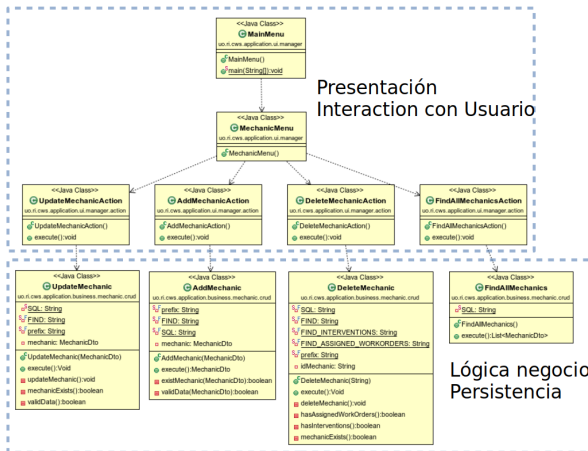
- **UI.** Clases `ui.***.action` implementan **interacción con usuario**, validaciones de datos, e invocan clases de negocio. Paquetes por actores.
- **Business.** Lógica de negocio + persistencia. Paquetes **por entidad**.

```

▼ 🏠 > uo.ri
  ▼ 🏠 > cws.application
    ▼ 🏠 > service.mechanic
      ▼ 🏠 > crud
        > 📄 > AddMechanic.java
        > 📄 > DeleteMechanic.java
        > 📄 > FindMechanicById.java
        > 📄 > FindMechanicByNif.java
        > 📄 > ListAllMechanics.java
        > 📄 > UpdateMechanic.java
        > 📄 > MechanicDto.java
      ▼ 🏠 > ui
        > 🏠 > cashier
        > 🏠 > foreman
        ▼ 🏠 > manager
          > 🏠 > contracts
          ▼ 🏠 > mechanic
            ▼ 🏠 > action
              > 📄 > AddMechanicAction.java

```

Diagrama clases Gestión de Mecánicos (CRUD) ²



²En los siguientes diagramas UML, podría haber algunos errores tipográficos relacionados con los nombres de paquetes, clases o métodos.

Refactorización clases

Escribir código AddMechanicAction tras la refactorización

```
public class AddMechanicAction implements Action {
```

```
    private static String SQL = "insert into TMechanics ("
        + "id, dni, name, surname, version) values (?, ?, ?, ?, ?)";
    private static final String URL = "jdbc:hsqldb:hsq://localhost";
    private static final String USER = "sa";
    private static final String PASSWORD = "";
```

```
    @Override
    public void execute() throws BusinessException {
```

```
        // Get info
        String dni = Console.readString("Dni");
        String name = Console.readString("Name");
        String surname = Console.readString("Surname");
```

```
        // Process
        Connection c = null;
        PreparedStatement pst = null;
        ResultSet rs = null;
```

```
        try {
            c = DriverManager.getConnection(URL, USER, PASSWORD);

            pst = c.prepareStatement(SQL);
            pst.setString(1, UUID.randomUUID().toString());
            pst.setString(2, dni);
            pst.setString(3, name);
            pst.setString(4, surname);
            pst.setLong(5, 1L);

            pst.executeUpdate();

        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
        finally {
            if (rs != null) try { rs.close(); } catch (SQLException e) { /* ignore */ }
            if (pst != null) try { pst.close(); } catch (SQLException e) { /* ignore */ }
            if (c != null) try { c.close(); } catch (SQLException e) { /* ignore */ }
        }
    }
```

```
    // Print result
    Console.println("Mechanic added");
```

```
public class AddMechanic {
```

```
    private static String SQL = "insert into TMechanics ("
        + "id, dni, name, surname, version) values (?, ?, ?, ?, ?)";
    private static final String URL = "jdbc:hsqldb:hsq://localhost";
    private static final String USER = "sa";
    private static final String PASSWORD = "";
```

```
    private MechanicDto dto;
    private Connection c = null;
```

```
    public AddMechanic ( MechanicDto arg ) {}
```

```
    public MechanicDto execute() throws BusinessException {
```

```
        PreparedStatement pst = null;
        // Process
        try {
            try {
                c = DriverManager.getConnection(URL, USER, PASSWORD);
                pst = c.prepareStatement(SQL);
                String id = UUID.randomUUID().toString();
                pst.setString(1, id);
                pst.setString(2, dto.dni);
                pst.setString(3, dto.name);
                pst.setString(4, dto.surname);
                pst.setLong(5, 1L);
                pst.executeUpdate();
            }
```

```
            dto.id = id;
            return dto;
        } finally {
            pst.close();
            c.close();
        }
    }
    catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

Create invoice class diagram

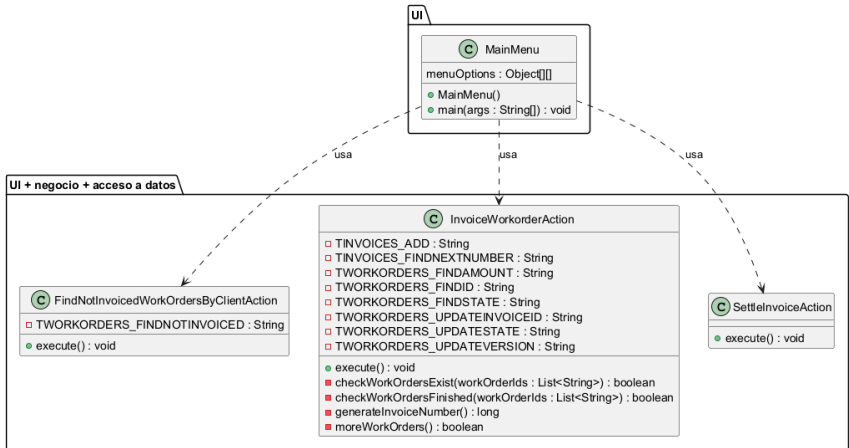
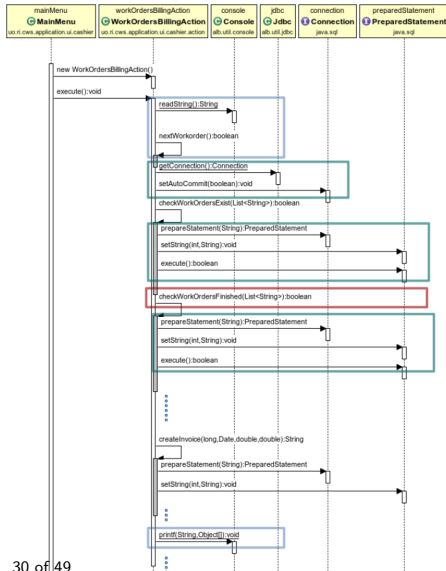


Diagrama de secuencia de InvoiceWorkorder



InvoiceWorkOrdersAction::execute()

```
// type work order ids to be invoiced in the invoice
do {
    String id = Console.readString("Type work order ids: ");
    workOrderIds.add(id);
} while ( nextWorkorder() );

try {
    connection = Jdbc.getConnection();

    if (! checkWorkOrdersExist(workOrderIds) )
        throw new BusinessException ( "Workorder does not exist");
    if (! checkWorkOrdersFinished(workOrderIds) )
        throw new BusinessException ( "Workorder is not finished

    long numberInvoice = generateInvoiceNumber();
    Date dateInvoice = Dates.today();
    // vat not included
    double amount = calculateTotalInvoice(workOrderIds);
    double vat = addVat(amount, dateInvoice);
    // vat included
    double total = amount * (1 + vat/100);
    total = Round.twoCents(total);

    long idInvoice = createInvoice(numberInvoice, dateInvoice, vat, total);
    linkWorkordersToInvoice(idInvoice, workOrderIds);
    markWorkOrderAsInvoiced(workOrderIds);

    displayInvoice(numberInvoice, dateInvoice, amount, vat, total);

}
catch (Exception e) {
    throw new RuntimeException(e);
}
finally {
    Jdbc.close(connection);
}
```

Azul interacción
con el usuario

Rojo lógica de
negocio

Verde acceso a datos

Ejercicio: Refactorizar update mechanic

- Señalar, en UpdateMechanic, interacción con el usuario, lógica de negocio y acceso a datos
- Dependencias.
- Revisar el nuevo diagrama de clases.
- Escribir el nuevo código.

Section 5

Fachada

¿Qué problema queda después de aplicar Capas?

- El cliente **crea instancias** de las **clases que implementan** los servicios y **llaman a sus métodos**.
- Debe conocer las **clases y métodos** que proporcionan las funcionalidades que necesita.
- Los cambios en la interfaz o en la forma de utilizar los servicios pueden afectar al cliente.

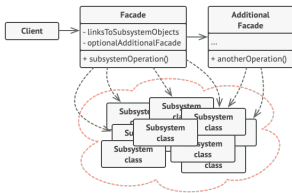
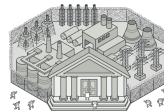
Calidad afectada: mantenibilidad.

Objetivo: proporcionar un **punto de acceso simplificado** al subsistema.

El patrón Fachada

Es un **patrón de diseño** análogo a una fachada en arquitectura.

Proporciona un **interfaz sencilla y unificada** a un **sistema complejo** (conjunto de objetos existentes).



- Los clientes interactúan con el subsistema a través de la fachada pero **ésto no impide el acceso directo a las clases del subsistema.**
- Podría haber **más de una fachada** para el mismo subsistema.

Ventajas

- **Menor acoplamiento:** el cliente deja de conocer y depender directamente de las clases que forman el subsistema.

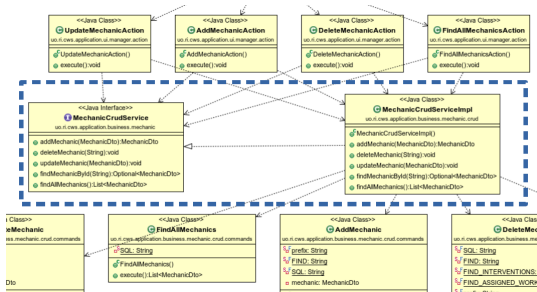
```
public class AddMechanicAction ... {  
    public void execute ( ) {  
        ...  
        AddMechanic a = new AddMechanic();  
        a.execute(nif, name, ...);  
    }  
}  
  
public class AddMechanicAction ... {  
    public void execute ( ) {  
        ...  
        MechanicFacade f = new MechanicFacade();  
        f.addMechanic(nif, name, ...);  
    }  
}
```

- **Simplicidad:** proporciona un punto de acceso único y de alto nivel a las funcionalidades del subsistema.
- **Mantenibilidad:** los cambios en la implementación del subsistema quedan localizados en la fachada. No afectan al cliente.

```
public class AddMechanicAction ... {  
    public void execute ( ) {  
        ...  
        AddMechanic2 a = new AddMechanic2();  
        a.execute(nif, name, ...);  
    }  
}  
  
public class AddMechanicAction ... {  
    public void execute ( ) {  
        ...  
        MechanicFacade f = new MechanicFacade();  
        f.addMechanic(nif, name, ...);  
    }  
}
```

Diagrama de clases

- **Cliente:** Sistema que interacciona con la fachada.
- **Interfaz fachada:** Declara las operaciones que ofrece al cliente.
- **Implementación interfaz:** Implementa las operaciones y accede a las clases del subsistema.
- **Subsistema:** Clases que implementan su funcionalidad.



Patrón Fachada en el caso de uso Gestión de Mecánicos

- Crear una **interfaz** (MechanicService).
- **Implementar** interfaz (MechanicServiceImpl)
- Identificar quién es quién en el diagrama de clases anterior.

```

▼ 📁 > uo.ri
  ▼ 📁 > cws.application
    ▼ 📁 > service.mechanic
      ▼ 📁 > crud
        > 📄 > AddMechanic.java
        > 📄 > DeleteMechanic.java
        > 📄 > ListAllMechanics.java
        > 📄 > ListMechanicById.java
        > 📄 > ListMechanicByNif.java
        > 📄 > UpdateMechanic.java
        > 📄 > MechanicDto.java
      ▼ 📁 > ui
        > 📄 > cashier
        > 📄 > foreman
        ▼ 📁 > manager
          > 📄 > contracts
          ▼ 📁 > mechanic
            ▼ 📁 > action
              > 📄 > AddMechanicAction.java
              > 📄 > DeleteMechanicAction.java
              > 📄 > ListAllMechanicsAction.java
              > 📄 > ListMechanicByNifAction.java
              > 📄 > UpdateMechanicAction.java
            > 📄 > MechanicMenu.java

```

Diagrama de clases

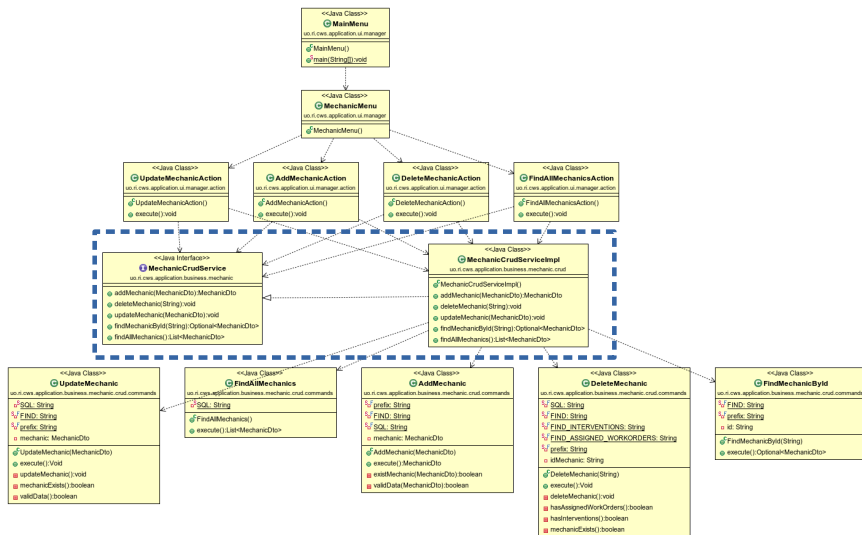
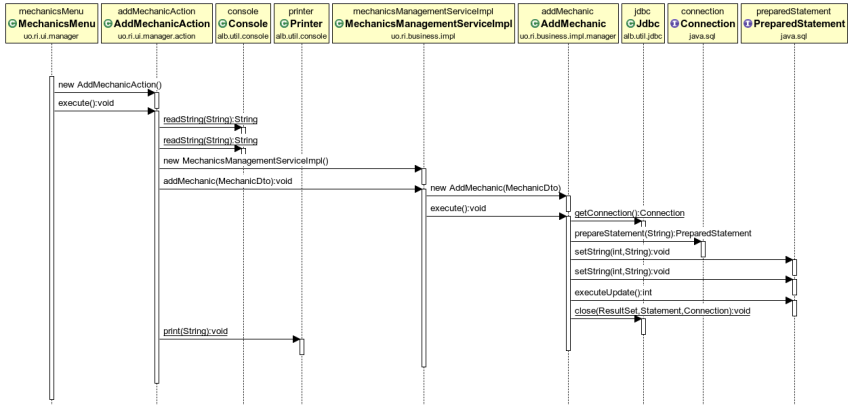


Diagrama de secuencia



Refactorizar las clases

- Escribe el código de las clases `Action`, `commands`, `MechanicService` y `MechanicServiceImpl`.
- Dependencias (`imports`)

Section 6

Factoría Simple

Problemas después de aplicar Fachada

Fachada ha desacoplado la capa de interacción del uso de los servicios, pero todavía **no de su creación**.

```
MechanicService s = new MechanicServiceImpl();  
s.addMechanic();
```

Aunque se utilice la interfaz MechanicService, el cliente **debe conocer e instanciar la implementación concreta**.

- Dependencia de MechanicServiceImpl.
- Los cambios de implementación pueden afectar al cliente.

Calidad afectada: mantenibilidad.

Objetivo: **encapsular la creación de objetos** para que el cliente no dependa directamente de sus implementaciones concretas.

Patrón Simple Factory

Una **Simple Factory** encapsula la creación de objetos, ocultando al cliente qué clase concreta se instancia.

Sin Factory



Con Factory



La factoría **suele** exponerse mediante métodos estáticos.

La factoría **no elimina el acoplamiento** con `MechanicServiceImpl`; **lo concentra** en un único lugar.

Ventajas

- **Desacoplamiento:** el cliente no depende de la implementación concreta, sino de su interfaz.
- **Centralización:** la creación de objetos y la dependencia con sus implementaciones quedan localizadas en la factoría.

Estructura de paquetes Mechanics Management con Simple Factory

1. UI solicita una clase service (getMechanicService) a ServiceFactory
2. ServiceFactory retorna una instancia de una clase que implemente la interfaz del servicio solicitado (MechanicServiceImpl).
3. Las clases UI conocen ServiceFactory y las interfaces de los services (MechanicService); no las clases que implementan el servicio.

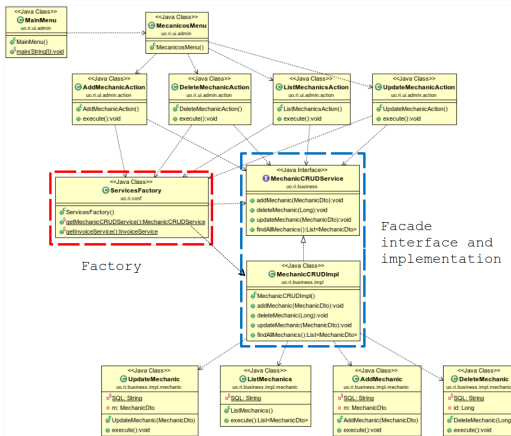
```

  ▾ > cws.application
    ▾ > service
      ▾ > mechanic
        ▾ > crud
          ▾ > commands
            > AddMechanic.java
            > DeleteMechanic.java
            > FindAllMechanics.java
            > FindMechanicById.java
            > FindMechanicByNif.java
            > UpdateMechanic.java
            > MechanicServiceImpl.java
            > MechanicService.java
          > ServiceFactory.java
        ▾ > ui
          > cashier
          > foreman
          ▾ > manager
            > contracts
            ▾ > mechanic
              ▾ > action
                > AddMechanicAction.java
                > DeleteMechanicAction.java
                > ListAllMechanicsAction.java
                > ListMechanicAction.java
                > UpdateMechanicAction.java
              > MechanicMenu.java

```

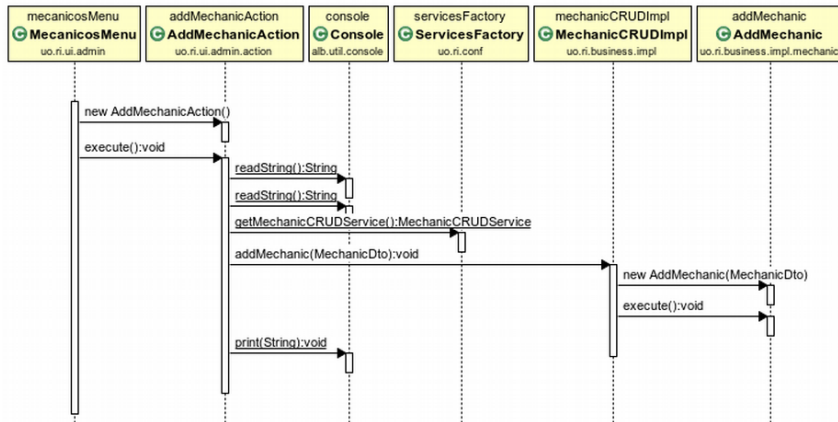
Diagrama de clases

Escribir código de AddMechanicAction



- ServiceFactory retorna una fachada.
- Cambiar la implementación del servicio sólo implica cambiar el método getMechanicService.

Diagrama de secuencia



Refactorizar el código

- Escribe el código de `ServiceFactory`.
- En los tipos `Action`, `MechanicService`, y `MechanicServiceImpl`, identifica los cambios y reescribe el código.
- Dependencias (imports).